

AI-Based OMR Sheet Evaluation



Submitted by:

M. Mateen Bashir / 23108125
Habiba Bibi / 23108111

Submitted to:

Mr. Muhammad Azhar

**Department of Robotics & AI
SZABIST University Islamabad
Spring 2026**

Table of Contents

1. Introduction	3
2. Features.....	3
3. Functional Requirements	4
4. Feasibility	5
5. Implementation.....	6
6. Conclusion	7
7. Contributions	7

1. Introduction

Objective examinations often use bubble answer sheets because they are quick to distribute and easy to process in large numbers. In practice, grading those sheets by hand is slow and error-prone, while specialized OMR hardware is not always available. This project addresses that gap with an AI-based OMR sheet evaluation system that runs on a normal laptop without a GPU, making it suitable for a typical Computer Vision lab environment and classroom demonstration. The system takes scanned or photographed OMR images, determines where each bubble should appear using a geometric template and optional perspective calibration, and extracts small image regions around every option. A convolutional neural network built with transfer learning on a lightweight backbone (such as MobileNetV2) then classifies whether each bubble is marked, which is more robust than relying only on simple pixel counts. The predicted answers are compared against a CSV answer key matched by Image_ID, and the tool reports accuracy, marks, confidence, and flags for ambiguous cases (for example multiple marks or low confidence). Supporting scripts handle training and batch evaluation, while a Streamlit web interface provides an interactive way to upload sheets, inspect results, and download outputs. Together, these pieces show how classical OpenCV-based processing and modern deep learning can be combined into a complete, practical tool for automated objective grading.

2. Features

Robust image processing

The system applies grayscale conversion, filtering, thresholding, and related OpenCV steps so bubble regions are more stable under noise, uneven lighting, and mild blur before patches are sent to the CNN.

Geometric alignment and templates

Bubble positions are anchored using a saved template and optional perspective calibration. This reduces error from skewed photos or perspective distortion—common when sheets are captured by phone or camera instead of a flatbed scanner.

Deep learning with transfer learning

A lightweight CNN built on a pretrained MobileNetV2 backbone (typically frozen) classifies whether each bubble is marked. This improves robustness compared to naive pixel-count rules while remaining feasible on CPU.

Synthetic and real training data

Synthetic bubble images support rapid bootstrapping and augmentation; real labeled patches from dataset splits improve realism. Together they make the training story complete for an academic ML + CV project.

Ambiguity detection and confidence

The pipeline identifies multiple marks, blanks, and low-confidence cases so those questions can be flagged for review rather than silently forced to a single letter grade outcome.

Rich outputs and analytics

For each sheet, the tool can emit predictions CSV, overlay images, audit JSON, and update a central student results file. Analytics include accuracy, optional negative marking, section-wise breakdowns, and template drift awareness.

Batch processing and web UI

Scripts support batch grading; the Streamlit frontend supports uploads, tables, metrics, overlays, and downloads—useful for presentation and quick manual verification.

3. Functional Requirements

Input and identification

The system must accept common raster formats (e.g., JPG/PNG) and map each file to an Image_ID that matches a row in the answer key (for example, deriving 122 from 122.jpg).

Answer key integration

The system must load answer_key.csv with columns such as Image_ID and Q1...Q30, then compute per-question correctness and summary scores.

Template and calibration persistence

The system must read saved configuration (template points, calibration data) so the same physical sheet layout can be evaluated reproducibly across sessions and machines.

Model inference

The system must load the trained CNN from models/ and classify bubble patches at evaluation time. Missing models or environment issues must fail with clear errors, not hidden fallback behavior.

Reporting and artifacts

Each evaluation must produce structured outputs: predictions, visual overlay, audit metadata, and aggregated student records suitable for archiving and instructor review.

Dual interfaces with shared core

CLI scripts and the Streamlit app must call the same core engine so results stay consistent whether the user runs a terminal command or uses the browser UI.

CPU-oriented performance

The design must remain practical on non-GPU laptops for typical lab-scale batches, using efficient preprocessing and a compact transfer-learning setup.

4. Feasibility

Technical feasibility

OpenCV is well suited to preprocessing, circle detection, and geometric transforms. TensorFlow/Keras is standard for transfer learning. Pandas simplifies CSV grading. Combined, these reduce technical uncertainty for a semester-scale project.

Hardware feasibility

The CNN operates on small patches and a bounded grid (questions $\times$ options), so CPU inference is acceptable for demonstration workloads. Training is heavier but runs offline and does not need to repeat for every new sheet once a model is saved.

Data feasibility

Synthetic data makes early development and demos possible even when real labels are limited. Real patches improve generalization; transfer learning helps when the labeled set is not huge.

Operational feasibility

Outputs are portable (CSV/JSON/PNG), which supports submission packaging, instructor review, and screenshot-based evidence in reports.

Limitations and mitigations

Accuracy depends on print quality, lighting, skew, and template fit. Mitigations include calibration wizards, exact template mapping, drift indicators, and confidence-based review queues. For high-stakes exams, flagged items should still be checked by a human.

5. Implementation

Technology stack

Implementation uses Python, OpenCV, NumPy, Pandas, TensorFlow (version aligned with the chosen Python release), Streamlit, and dependencies declared in requirements.txt.

Software architecture

Core logic lives under src/omr/: configuration, OMR engine, CNN module, calibration utilities, and template helpers. scripts/ contains training, single-sheet evaluation, batch runs, and interactive calibration/template tools. models/ stores weights and JSON configuration; outputs/ stores run artifacts. app.py implements the Streamlit frontend on top of the same evaluation functions used by the CLI.

Runtime evaluation pipeline

A sheet image is loaded and preprocessed; template and calibration map expected bubble centers into image coordinates; optional refinement (e.g., circle snapping, local alignment) reduces small positional error; patches are extracted and resized; the CNN scores each option; decision rules resolve ambiguous cases; predictions are compared to the matching answer-key row; CSV/JSON/overlay files are written and student summaries are updated.

Offline training pipeline

The CNN is trained with synthetic augmentation and optional real labeled data. The trained model is saved under models/ for reuse by both scripts and the web app.

Verification strategy

Development verification combines quantitative checks against the answer key with qualitative checks using overlay images, ensuring that good scores correspond to visibly correct bubble alignment.

6. Conclusion

This project demonstrates a workable CPU-oriented pipeline for automated OMR evaluation that combines image processing and geometry with transfer-learning-based bubble classification. By including calibration and template tooling, structured reporting (CSVs, overlays, audit metadata), batch processing, and a web frontend, the deliverable goes beyond a single script and reads as a complete software system. The main takeaway is that strong results depend not only on the neural network but also on reliable alignment between the physical sheet and the digital template, which is why calibration, drift awareness, and confidence-based review are important parts of the design. The system is well suited as a Computer Vision project that balances OpenCV skills with machine learning practice; future work could extend it with better dewarping, support for multiple layouts, and continuous improvement from human-corrected review items for deployment in more demanding real-world conditions.

7. Contributions

Name	Registration	Contribution
M. Mateen Bashir	23108125	Code/Documentation
Habiba Bibi	23108111	Code/PPT